Project Report

Team7

SkyRise

Introduction

Booking a flight can often be a challenging and time-consuming task, with many steps involved that make it feel complicated and overwhelming. In a world where everything is becoming digital, it makes sense to simplify flight booking as well.

Our goal is to develop a web application for an airline company that offers a simple and smooth way for users to book flights, track their flight details, provide feedback, and much more! We aim to enhance the overall user experience and meet the expectations of modern travelers by providing a user-friendly interface and simplifying the entire process.

Functional Requirements:

Users Requirements:

Sign Up Requirements

Function:	Account creation by users.
Description:	Users register by providing personal information, either manually or via their Gmail account. Ensures data integrity with client-side and server-side validation.
Inputs:	Email Address, Password, Confirm Password, Country Code, Phone Number, National ID, Date of Birth, First Name, Last Name, Passport Number, Issuing Country
Source:	User-provided through input fields on the registration form.
Outputs:	User account created, stored in the database; confirmation email sent.
Action:	1. Users can choose to sign up using Gmail for faster registration. 2. Input fields are provided for each required detail. 3. Upon submission, client-side validation checks for: a. Correct email format and uniqueness b. Password length and complexity, and matching confirmation password c. Valid national ID and phone number formats d. Minimum age requirement 4. Server-side validation on submission, with appropriate feedback if any validation fails.
Requirements:	Input fields for text, password fields, dropdowns, date pickers, and a submit button to initiate registration after validation.
Pre-condition:	None.
Post-condition:	Successful account creation; user directed to login or home page.
Side effects:	• Sends a welcome email upon successful account creation. • If Gmail is used, some fields may be auto-filled.

Login System Requirements

Function:	User authentication for access.
Description:	Enables users to log in by entering their email and password. Provides real-time feedback on invalid inputs and verifies credentials with client-side and server-side validation. Redirects users to the appropriate page based on role upon successful login.
Inputs:	Email and Password.
Source:	User input on the login form.
Outputs:	A message indicating success or failure of the login attempt, and if successful login, redirection to the main website (user) or admin dashboard (admin) is made.

CamScanner

Action:	1. Client-side validation for email format and password complexity. 2. Server-side validation: a. If valid, display success message and redirect user. b. For admins, redirect to dashboard; for users, redirect to home page. c. Invalid credentials result in an error message.
Requirements:	Input fields for text (email) and password, with a submit button.
Pre-condition:	User must have an existing account.
Post-condition:	User is logged in and redirected accordingly.
Side effects:	None.

Password Reset Requirements

Function:	Allows users to reset their password via email.
Description:	Enables users to initiate a password reset by entering their email to receive a reset link, allowing the user to set a new password and regain account access.
Inputs:	Email address to receive the password reset link.
Source:	User-provided through the password reset form.
Outputs:	Success or failure message depending on email validity, and in case of valid email, password reset email is sent.
Action:	1. Client-side validation for email format. 2. Server-side validation: a. If the email exists, send password reset email. b. If not, display an error.
Requirements:	Text input for email, and a submit button to initiate the reset, and the database contains registered user emails for validation.
Pre-condition:	User must remember the registered email address.
Post-condition:	If the email exists, a reset link is sent to the provided email. The user can use this link to reset the password.
Side effects:	A new password sent to the user, allowing them to log in and change it later.

Gmail Login System Requirements

Function:	Simplified login using Gmail credentials.
Description:	Allows users to initiate the login process by selecting a Gmail login option, which redirects them to Google for authentication. Upon successful authentication, users are redirected back to the main website or dashboard.
Inputs:	Gmail login button to initiate Google-based authentication.
Source:	User clicks Gmail icon to authorize login via Google.
Outputs:	Login authenticated by Gmail, user redirected to the main website (user) or admin dashboard (admin).
Action:	1. Button redirects user to Google API for authentication. 2. Redirection: a. On successful authentication, redirects the user to the main website or dashboard. b. If authentication fails, displays an error message and remains on the login page.
Requirements:	Button to initiate Gmail authentication.
Pre-condition:	User has an active Gmail account.
Post-condition:	User is authenticated via Gmail and redirected to the appropriate page on the website.
Side effects:	None.

Change Password Requirements

CamScanner

Function:	Enables users to update their password.
Description:	Allows users to update their password by first verifying their old password, then setting a new password with validation checks on complexity and matching confirmation. Provides feedback at each step of the process.
Inputs:	1. Old Password (First screen). 2. New Password and Confirm New Password (Second screen).
Source:	User-provided in the password change form.
Outputs:	Password updated in the system.
Action:	1. First screen: Client-side validation of old password complexity. 2. If valid, server-side validation of old password, then proceed to the second screen. 3. Second screen: Validate new password format and matching confirmation password.
Requirements:	Input fields for old and new passwords and a submit button to finalize change.
Pre-condition:	User is logged in and remembers old password.
Post-condition:	New password saved in the database; old password invalidated.
Side effects:	None.

Flight Booking Search Functionality

Function:	Search for flights.
Description:	Provides a search functionality where users can specify departure and arrival locations, dates, flight legs, and the number of passengers. The system validates the input and queries the database to display matching flights.
Inputs:	Departure Location, Arrival Location, Departure Date, Return Date, Flight legs, and Number of Passengers
Source:	All input fields provided by the user through the search form.
Outputs:	List of flights that match the search criteria.
Action:	1. Client-side validation on dates, locations, and passenger count (at least one). 2. Server-side query to retrieve matching flights.
Requirements:	Dropdowns for locations, date pickers, and numeric input for passengers.
Pre-condition:	None.
Post-condition:	List of flights that matches the search criteria is displayed if found.
Side effects:	None.

Available Flights Display

Function:	Display available flights.
Description:	Allows users to view a list of available flights from the specified departure location to the arrival location. Users can navigate between pages.
Inputs:	Page number (numeric box).
Source:	User navigates through pages.
Outputs:	List of available flights based on the selected page number.
Action:	Server-side validation to retrieve flights of the chosen date.
Requirements:	Page navigation to change page number.
Pre-condition:	User has completed flight search
Post-condition:	System displays flights for the selected page.
Side effects:	None.

Reserving Ticket

CamScanner

Function:	Collect passenger information necessary for booking a ticket.
Description:	Allows users to enter details about each passenger, including name, age, passport number (if required), and contact information.
Inputs:	First Name, Last Name, Date of Birth, Passport Number, Nationality, Contact Information (email, phone number)
Source:	User-provided through the reservation form.
Outputs:	Confirms that passenger details have been successfully submitted and validated.
Action:	1. Client-side format validation for email and phone. 2. Server-side passport validation.
Requirements:	Text, numeric, and email inputs with a submit button.
Pre-condition:	User has selected a flight.
Post-condition:	Passenger details are validated and saved in the database.
Side effects:	None.

Flight Tracking

Function:	Allows users to track their booked flights.
Description:	Provides a list view of all flights booked by the user, displaying flight status (e.g., arrived, on time, canceled). Users can search for specific flights and apply filters to narrow down results.
Inputs:	Search Query containing Flight Number, Date of Departure, Departure Location, Destination, Status.
Source:	User provides search and filter details for flight tracking on the flight tracking page.
Outputs:	List of flights matching search criteria.
Action:	1. Client-side validation for correct formats. 2. Server-side search in the database.
Requirements:	Text boxes, date picker, and dropdowns for input and a submit button
Pre-condition:	User has an account with booked flights.
Post-condition:	Matching flights displayed.
Side effects:	None.

Feedback System

Function:	Collects feedback from users post-flight.
Description:	The system checks for flights associated with the user account. If a recent flight without feedback is found, a feedback prompt appears, allowing the user to rate the flight experience and leave comments.
Inputs:	Overall rating, Punctuality, Comfort, Service, Food/Beverage, Cleanliness, and Comments (optional)
Source:	User provides ratings and comments.
Outputs:	Feedback saved in the database, visible to the admin.
Action:	1. Validates that all required fields are completed. 2. Stores feedback in the database.
Requirements:	Star Rating, slider, text area, and submit button.
Pre-condition:	User has a recent flight eligible for feedback.
Post-condition:	Feedback saved in the database; popup disabled for this flight.
Side effects:	User-submitted feedback appears on the admin feedback page.

CamScanner

Admin Requirements:

Flights Viewing

Function:	View current flights with filters.
Description:	Allows admins to view current flights, filtered by source, destination, and status (complete/ incomplete).
Inputs:	Selected date, source, destination, and status filters.
Source:	Admin-specified filters.
Outputs:	Filtered list of flights displayed with a timeline view (7 days per page).
Action:	The filtering and sorting query is applied to the flights in the database. Display the returned flights. If no flights returned, display “No Flights”.
Requirements:	Combo boxes for locations, date pickers, Buttons for sorting, and filter buttons.
Pre-condition:	Existing flights in the system.
Post-condition:	Filtered flights are displayed.
Side effects:	None.

Flight Management

Airports Configuration.

Function:	Manage available airports and estimated flight durations.
Description:	Allows admins to configure airports and estimated flight durations between airport pairs.
Inputs:	List of available airports and estimated flight durations.
Source:	Admin-provided data.
Outputs:	Updated list of airports and flight durations displayed on the website.
Action:	Admin inputs airport list and durations, which are saved and made available for flight planning.
Requirements:	Accurate airport codes and realistic flight durations.
Pre-condition:	Admin access privileges.
Post-condition:	Airports and durations are successfully added to the system.
Side effects:	New flight additions auto-calculate arrival dates based on pre-set durations.

Add Flight

Function:	Add new flights to the system.
Description:	Provides a form for admins to input flight details and submit to add a new flight.
Inputs:	Departure and arrival airports, departure date/time, available seats, ticket prices, flight legs.
Source:	Admin-provided data through flight addition form.
Outputs:	Confirmation message and flight data added to the database.
Action:	Client-side and server-side validations are performed. If valid, flight details are stored and a confirmation message is sent. If not valid, an error message is sent.
Requirements:	Input fields for text, date pickers, and submit button.
Pre-condition:	Admin is logged in.
Post-condition:	New flight is available in the system, with unique flight number and arrival date/time calculated.
Side effects:	None.

CamScanner

Modify Flight

Function:	Update existing flight details.
Description:	Allows admins to modify flight details by selecting flights based on specific filters.
Inputs:	Departure and arrival airports, departure date/time, available seats, ticket prices, flight legs, modification reason, optional notes.
Source:	Admin-provided data through flight modification form.
Outputs:	Updated flight details in the database and confirmation message.
Action:	Client-side and server-side validations are performed. If valid, flight details are updated in the database and a confirmation message is sent. If not valid, an error message is sent.
Requirements:	Input fields for text, select box, date pickers, and submit button.
Pre-condition:	Admin has selected a flight to modify.
Post-condition:	Flight details are updated; passengers notified of changes to relevant fields.
Side effects:	Notifications sent to passengers if airports, dates, or prices are modified.

Cancel Flight

Function:	Cancel an existing flight.
Description:	Allows admins to cancel flights, providing a reason and optional notes.
Inputs:	Departure and arrival airports, departure date/time, available seats, ticket prices, flight legs, cancelation reason, optional notes.
Source:	Admin-provided data.
Outputs:	Confirmation message stating that the cancelation is done successfully.
Action:	Client-side validation is performed. If valid, send notification to all affected passengers.
Requirements:	Input fields for text, select box, date pickers, and submit button.
Pre-condition:	Admin has selected a flight to cancel.
Post-condition:	Flight status updated as "Canceled," and affected passengers are notified.
Side effects:	Email notifications sent to all affected passengers.

Upgrade users to admins

Function:	Manage user privileges.
Description:	Provides admins with the ability to view and upgrade users to admin status.
Inputs:	User's email to be upgraded.
Source:	Admin-selected user email.
Outputs:	Confirmation message of user status change.
Action:	1. Client-side validation for email format. If invalid, sends an error. 2. Server-side validation for email existence. If valid, success message is displayed. If not valid, failure message is displayed.
Requirements:	Input field for texts, and submit button.
Pre-condition:	Admin is logged in. Existing user profile in the system.
Post-condition:	User is upgraded to admin status.
Side effects:	None.

Archive

Function:	View archived flights.

CamScanner

Description:	Allows admins to filter and view past flights based on source and destination.
Inputs:	Source and destination filters.
Source:	Admin-specified filters.
Outputs:	List of archived flights matching filters.
Action:	The filtering query is applied and the returned flights are displayed.
Requirements:	Input field for texts, and submit button.
Pre-condition:	Admin is logged in. Existing archived flights in the system.
Post-condition:	Filtered archived flights are displayed.
Side effects:	None.

Feedback

Function:	Filter and manage user reviews based on star ratings and category performance.
Description:	Admin can filter reviews by star ratings (1 to 5 stars) and categories (Punctuality, Comfort, Service, Food & Beverage, Cleanliness). The system will display filtered reviews along with their detailed ratings and user comments. Admin can also view average ratings and total review count.
Inputs:	1. Star rating (1-5 stars). 2. Category performance filter (Excellent, Fair, Poor).
Source:	User reviews stored in the database. Ratings and category performance data are extracted from each review.
Outputs:	1. Average star rating of filtered reviews and total number of reviews that match the filter criteria. 2. List of filtered reviews, including: a. Star rating (1-5 stars). b. Category performance ratings (Punctuality, Comfort, Service, Food & Beverage, Cleanliness). c. User comments.
Action:	1. Admin applies filters by selecting star ratings and category performance. 2. The system retrieves and displays the relevant reviews in a paginated list (10 reviews per page). 3. Displays average star rating and total number of reviews.
Requirements:	1. Reviews must be tagged with star ratings and category performance scores (Punctuality, Comfort, Service, etc.). 2. Reviews must be stored in a database that allows querying and filtering based on multiple parameters (star rating and category performance).
Pre-condition:	1. Reviews are available in the system with ratings and performance feedback. 2. Filters are set by the admin.
Post-condition:	1. Reviews are filtered and displayed based on the selected criteria. 2. Average star rating and total number of reviews are updated.
Side effects:	None.

Overview

Function:	Display system statistics.
Description:	Provides a summary of users, reservations, and flights.
Inputs:	None.
Source:	System database.
Outputs:	Statistics on users, reservations, and flights.
Action:	System retrieves and displays relevant statistics.
Requirements:	None.
Pre-condition:	Existing data on users, reservations, and flights.

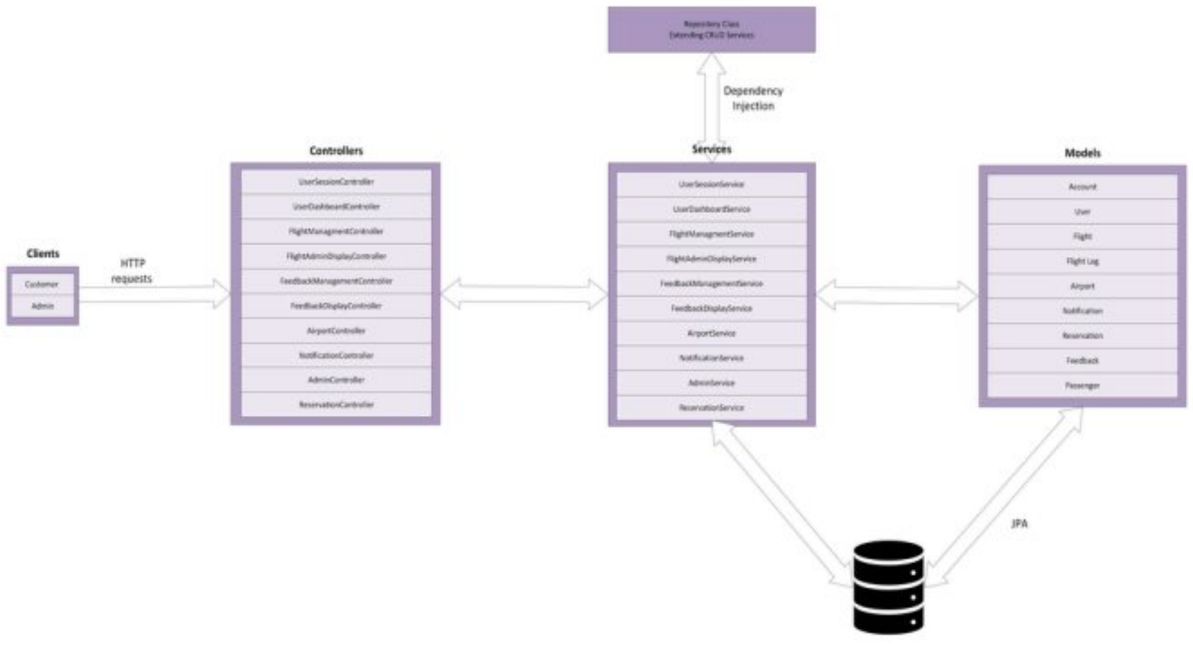
CamScanner

Post-condition:	Updated summary displayed on the dashboard.
Side effects:	None.

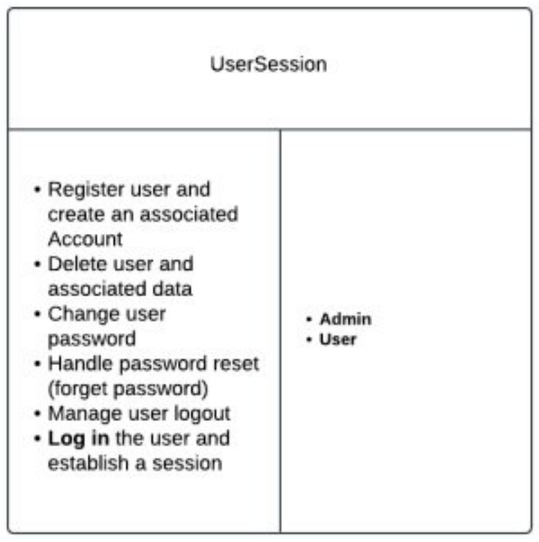
Non-Functional Requirements:

1. **Response Time**: The system should respond to user queries (e.g., searching for flights, booking a flight) within 2 seconds. (Apache JMeter)
2. **Throughput**: The system should be able to handle at least 10,000 concurrent users during peak times.
3. **Security**: user passwords must be hashed using strong password hashing algorithm. (Argon2, BCrypt).
4. **Timeout Duration**: The system will enforce a maximum booking timeout period of 10 seconds for flight reservations. If the booking process is not completed within this time frame, the system will automatically terminate the session and notify the user of the timeout.
5. **User Interface (UI) Design**: The interface should be intuitive and user-friendly, allowing booking admins, and customers to manage flights in the system without training.
6. **Ethical Requirements**: (Non-Discrimination) Ensure that the system does not discriminate against users based on, gender, nationality, or any other characteristic. The booking process should be equitable for all users.
7. **Reliability**: The system should consistently provide customers with accurate and up-to-date information regarding flight statuses, including departures, arrivals, delays, and cancellations

High Level/Architecture Diagram:



CRC:



CamScanner

NotificationManager	
• Notify users of flight cancellations or other relevant updates. • Allow users to delete specific notifications. • Coordinate with Admin to issue notifications to all users on a canceled flight.	• **Admin** • **User**

FlightService	
1- Get the number of remaining seats of a flight. 2- Change a flight status to completed. 3- Determine if a flight is available. 4- Check if the total weight of a flight passengers is with the allowed limit.	1- Flight 2- FloghtDao

Flight	
1- Store details about the flight. (arrival, and departure city, date, and airport) (current, and max number of passengers) 2- Check if the flight is full. 3- Store the weight information of a flight. (max weight, max weight per person)	

FligthDao	
1- find all flights by departure city, arrival city, departure time, arrival time, departure airport, or arrival airport. 2- get a flight max number of passengers. 3- get a flight current number of passengers.	1- Flight

ReservationService	
1- Create, delete, and retrieve reservations. 2- List reservations by UserID. 3- List reservations by FlightID.	1-ReservationDAO 2-ReservationDTO

ReservationDAO	
1- Add a new reservation to data storage. 2- Delete an existing reservation form data storage 3- Retrieve a reservation by ReservationID form data storage.. 4- List reservations by UserID, or FlightID form data storage.	1-ReservationDTO

ReservationDTO	
1- Hold reservation information such as ReservationID, FlightID, UserID, seat class, and passengers information. 2- Provide methods to retrieve reservation details.	1- Passenger

Passenger	
1- Store passenger information such as PassengerID, first name, last name, passport number, and date of birth. 2- Provide methods to retrieve passenger details.	1-ReservationDTO

CamScanner

FlightManagement	
• **Add a Flight:** Insert a new flight record into the flight table in the database. • **Update a Flight:** Modify details of an existing flight based on its ID. • **Cancel a Flight:** Mark a flight as canceled. • **Add Flight Legs:** Add multiple legs to a particular flight (useful for multi-stop flights).	• Flight • FlightLeg

Admin	
• **Upgrade User Role:** promote a user to an administrator. • **Get User and Reservation Counts:** Provide counts of users, reservations, and current flights for administrative reporting. • **Calculate Average Rating:** Retrieve and calculate the average rating of flights and services to monitor customer satisfaction.	• User • Reservation • FlightLeg • Feedback

FeedbackDisplay	
• **Retrieve All Feedback:** Fetch all feedback data with pagination. • **Filter Feedback:** Provide filtered feedback list based on specific criteria.	• Feedback

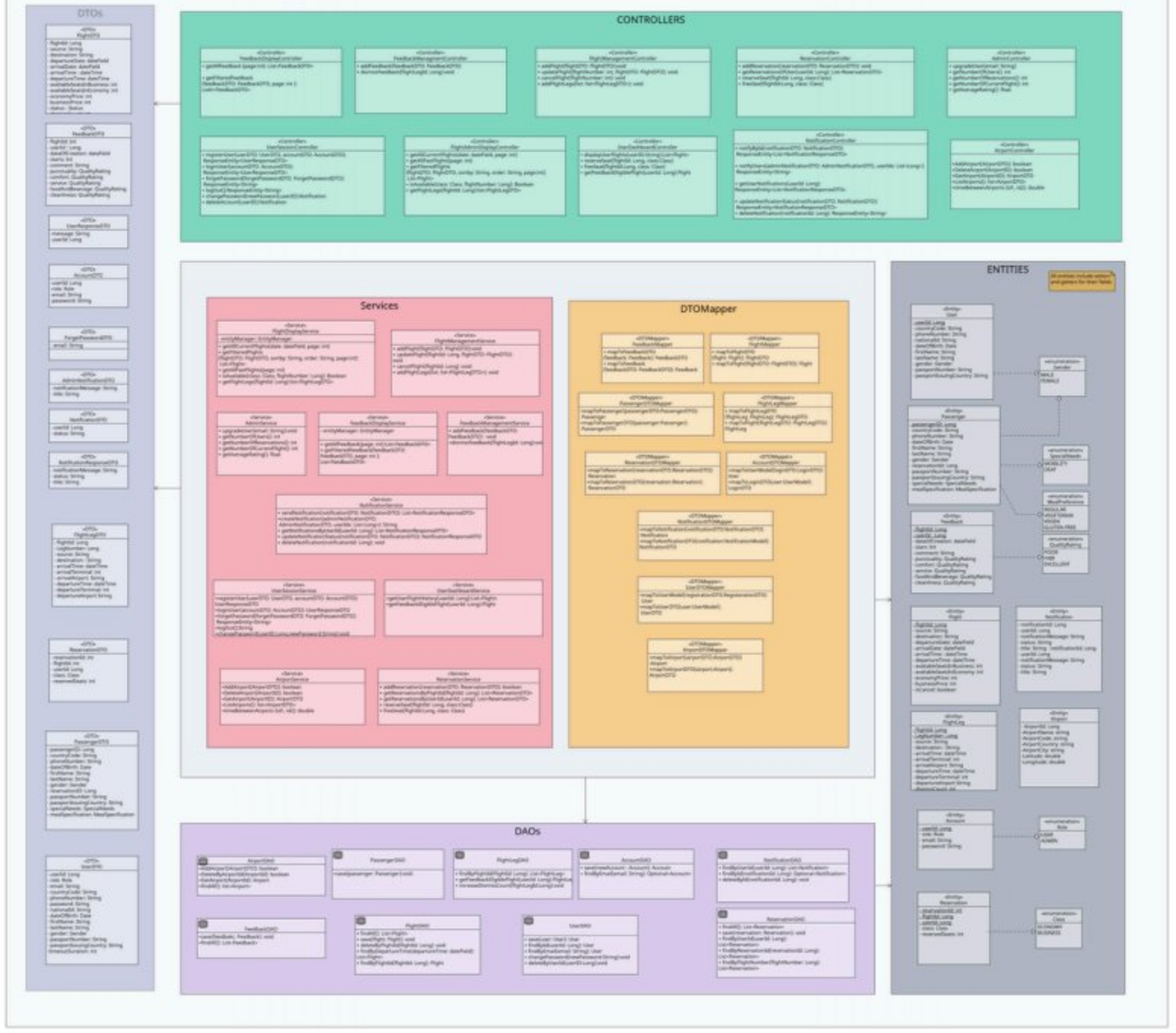
FeedbackManagement	
• **Add Feedback:** allow users to submit feedback for a flight and then save it to the database. • **Dismiss Feedback:** based on flightID it increases its dismiss count (reaching 3 makes feedback inaccessible).	• Feedback

UserDashboard	
• **Get User Flight History:** Retrieve a list of past flights associated with a specific user. • **Get Feedback Eligible Flights:** Identify flights for which the user can still provide feedback based on a recent timeframe and limited to flights that haven't been dismissed more than 3 times.	• Reservation • FlightLeg

CamScanner

FlightDisplay	
• **Retrieve Current Flights:** Fetch all current flights based on a given date and pagination. • **Filter and Sort Flights:** Provide a filtered and sorted list of flights according to specified criteria ,order, and page number. • **Retrieve Past Flights:** Fetch a list of past flights. • **Check Flight Availability:** Determine if it fully booked or not. • **Retrieve Flight Legs:** Return a list of flight legs for a specific flight (which might represent different segments in a flight with stops).	• Flight • FlightLeg

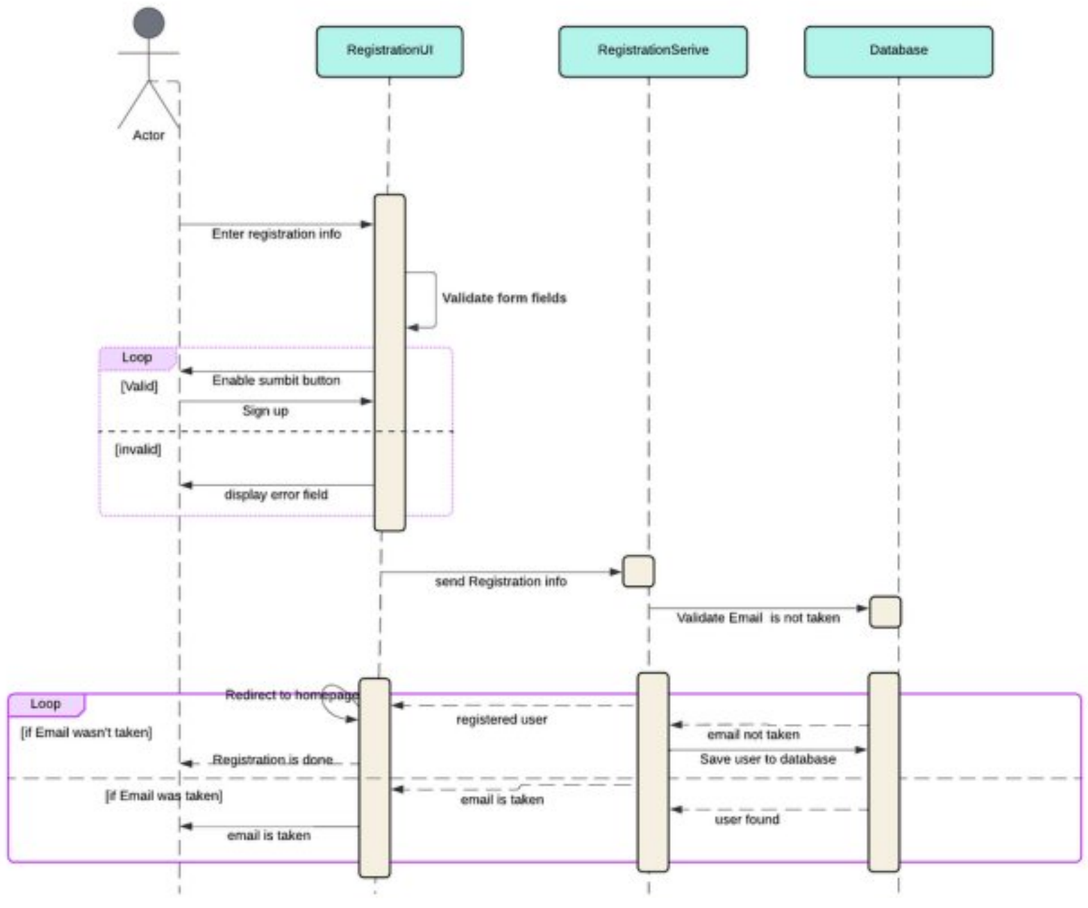
Class Diagram:



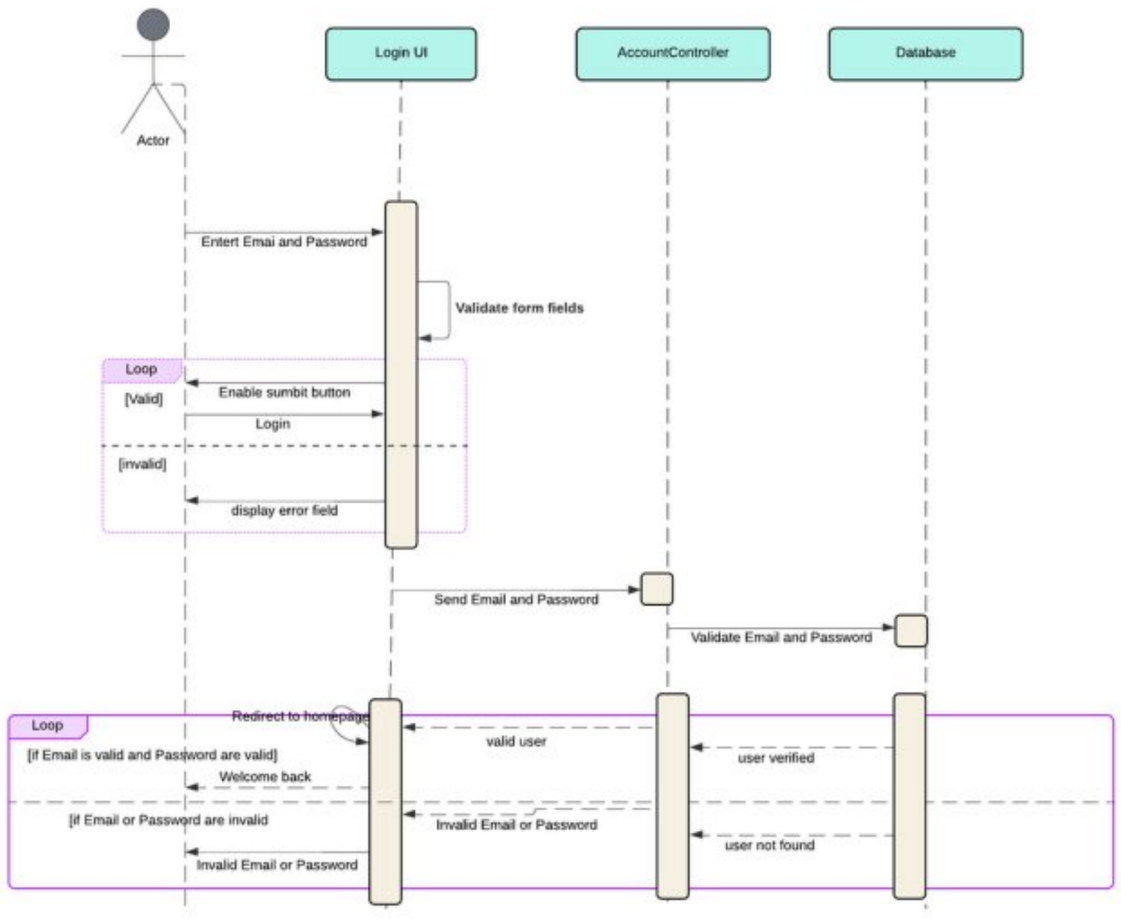
Interaction Diagram:

Registration:

CamScanner

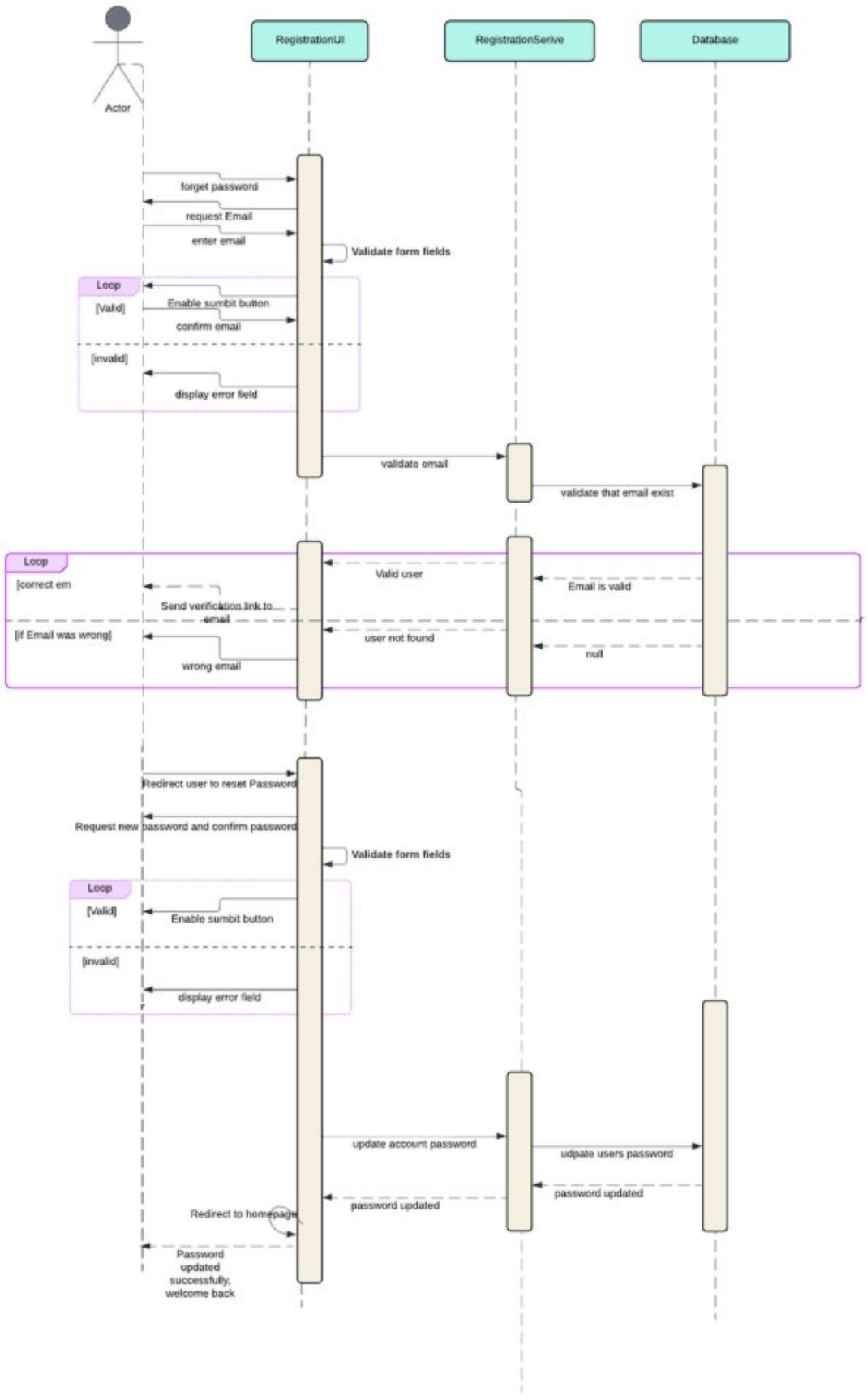


Login:



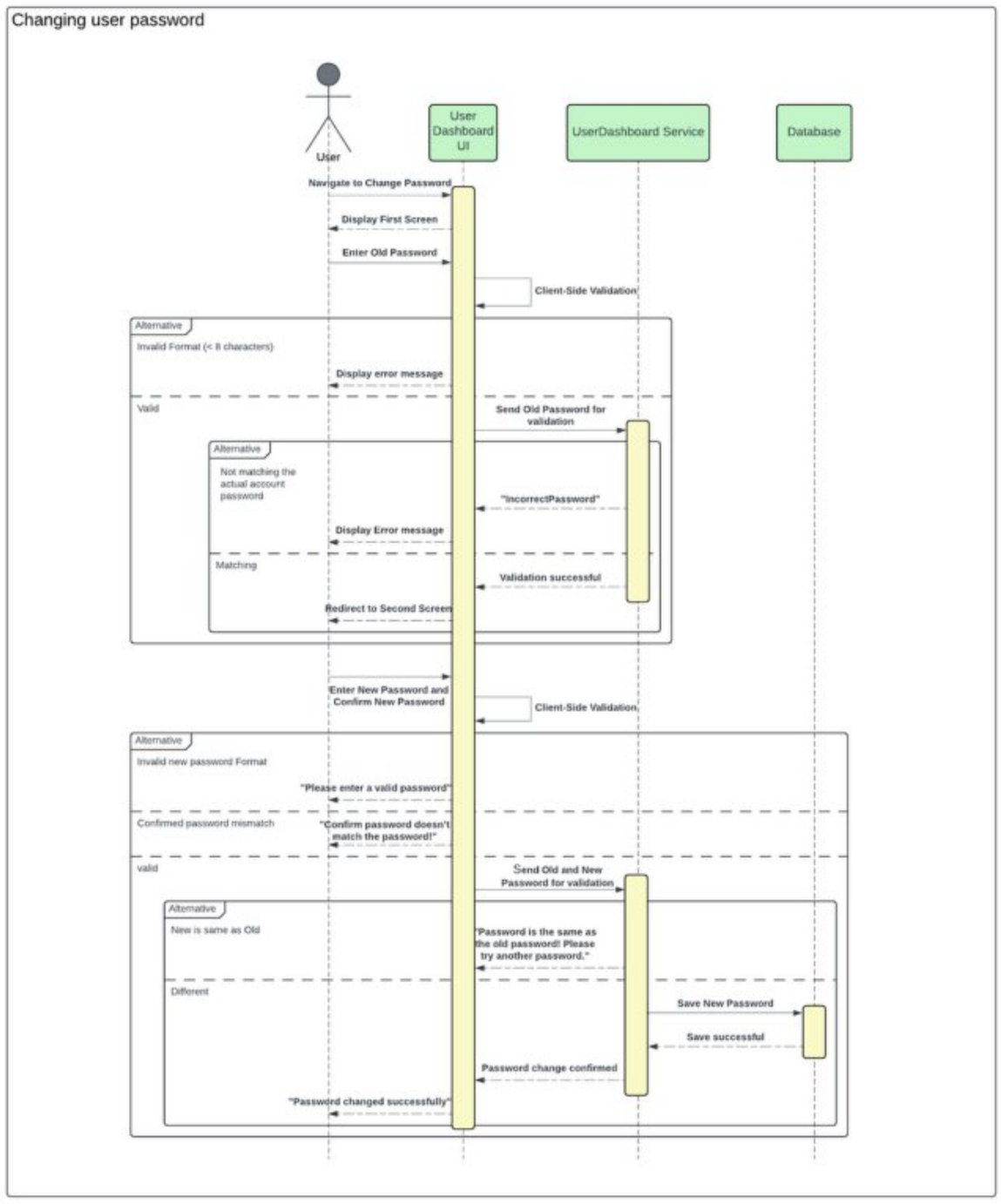
Forget Password:

CamScanner

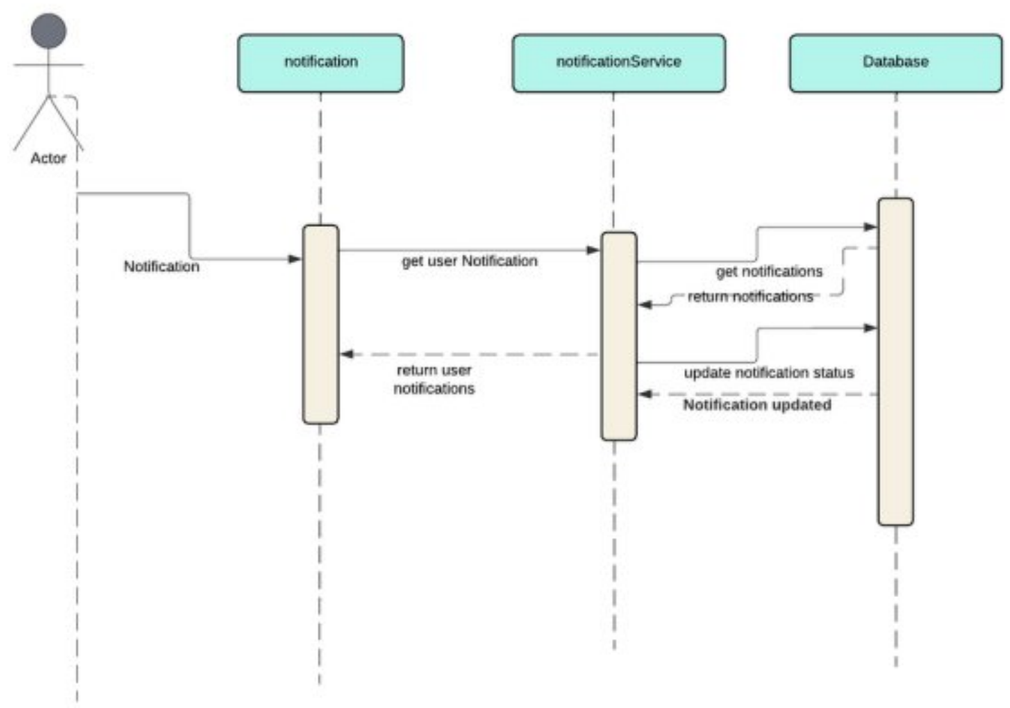


Changing User Password:

CamScanner

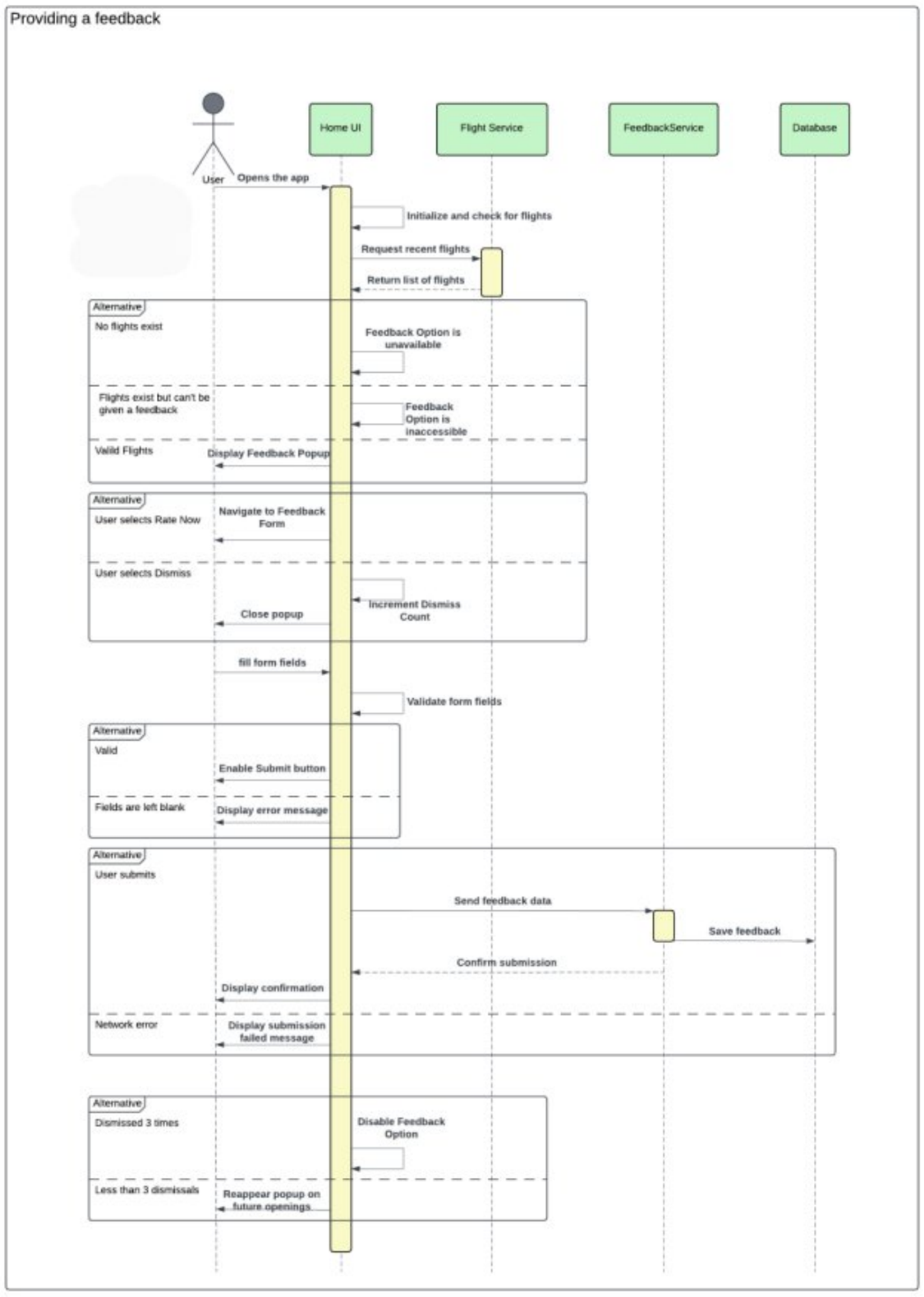


User notification:



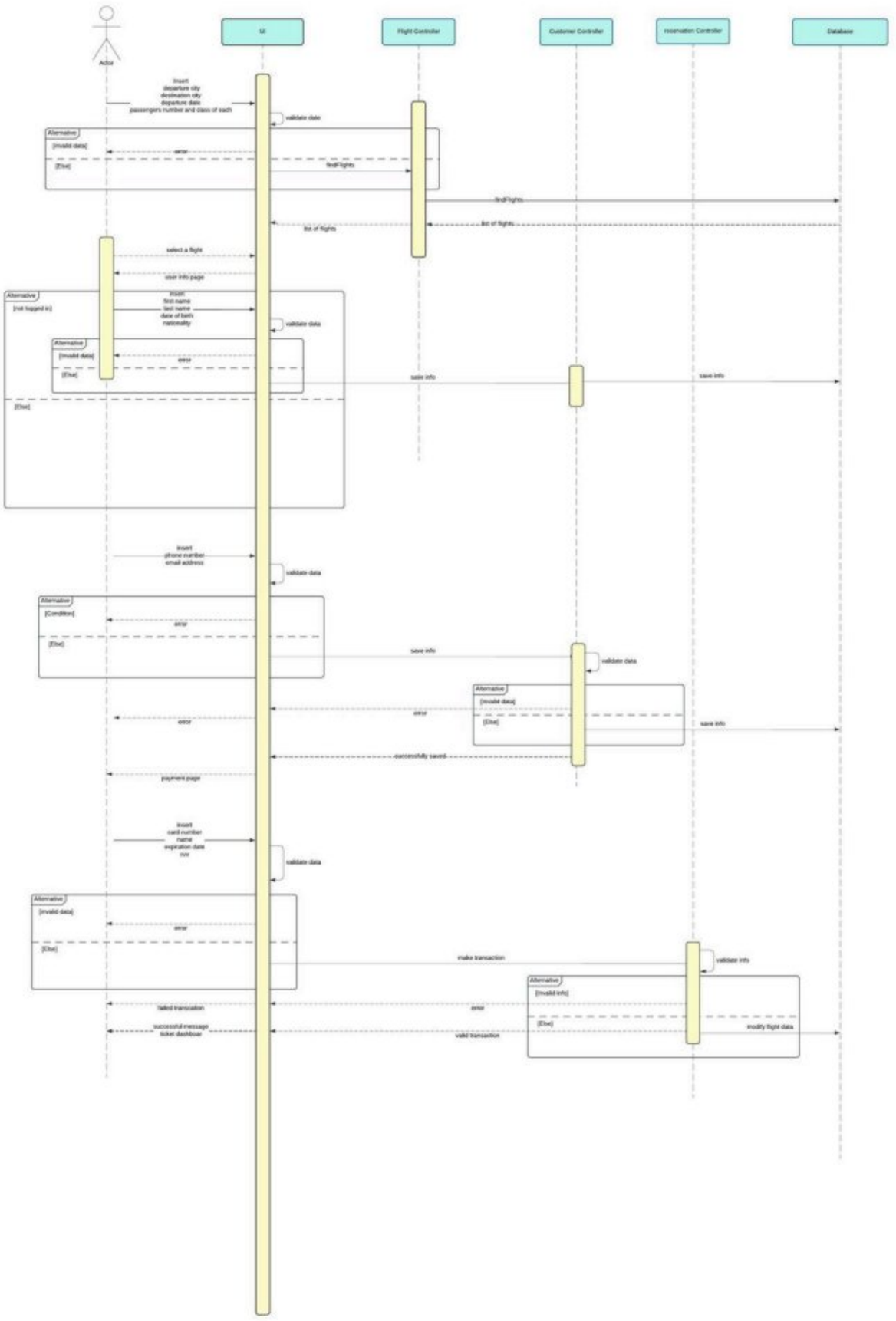
Providing a feedback:

CamScanner

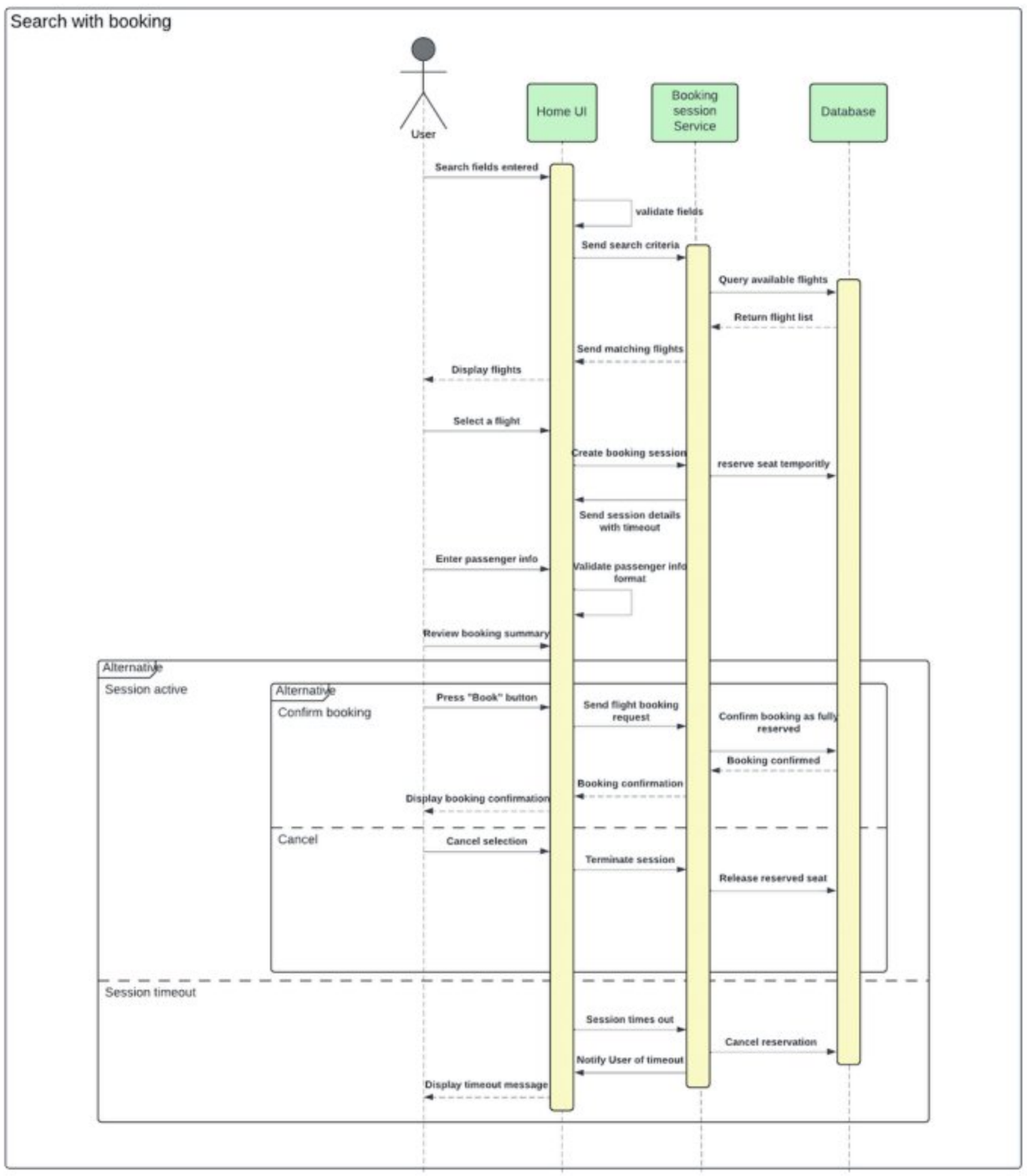


Search and Booking flights:

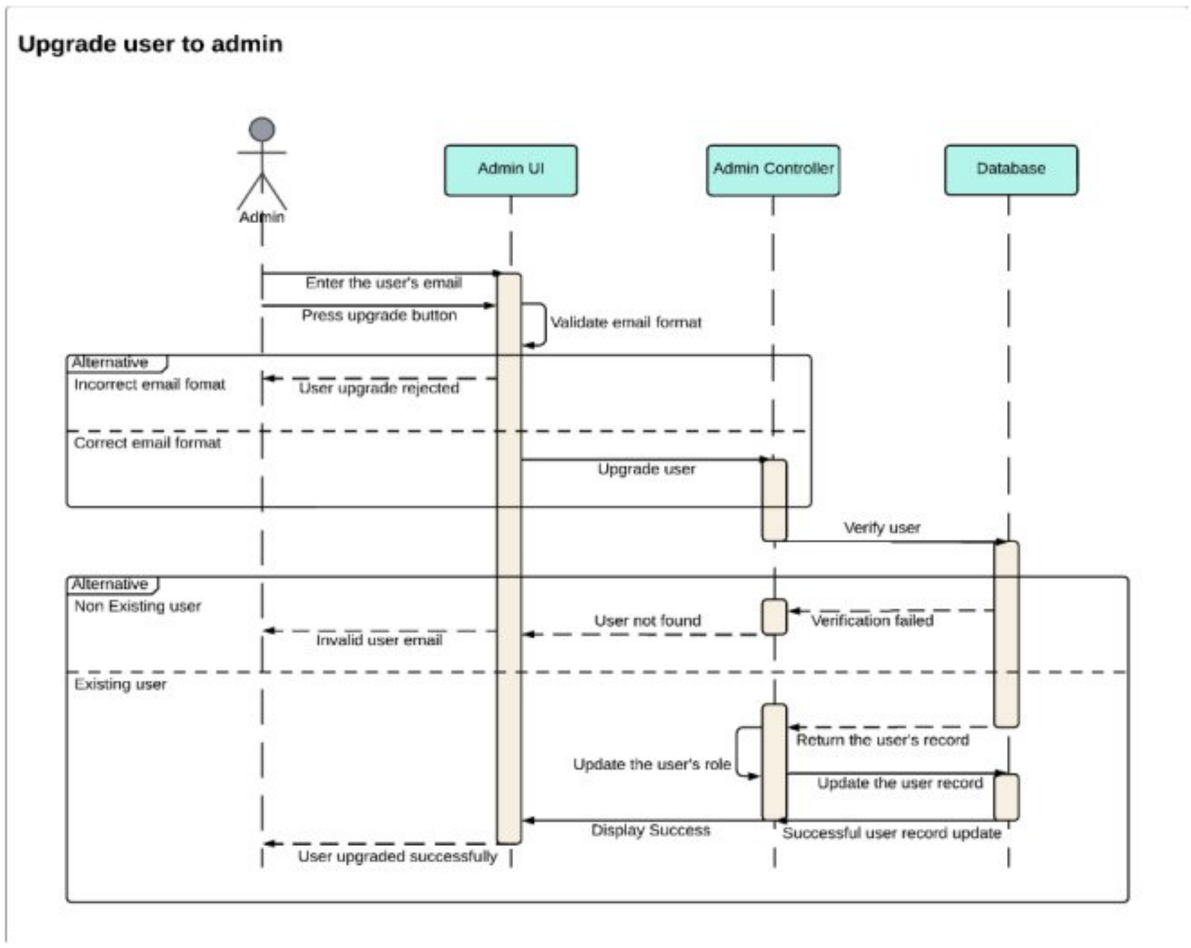
CamScanner


Actor
UI
Flight Controller
Customer Controller
reservation Controller
Database
insert
departure city
destination city
departure date
passengers number and class of each
validate date
Alternative
[invalid data]
error
[Else]
findFlights
findFlights
list of flights
list of flights
select a flight
user info page
Alternative
[not logged in]
insert
first name
last name
date of birth
nationality
validate data
Alternative
[invalid data]
error
[Else]
[Else]
save info
save info
insert
phone number
email address
validate data
Alternative
[Condition]
error
[Else]
save info
validate data
Alternative
[invalid data]
[Else]
error
save info
successfully saved
payment page
insert
card number
name
expiration date
cvv
validate data
Alternative
[invalid data]
error
[Else]
make transaction
validate info
Alternative
[invalid info]
[Else]
modify flight data
failed transcation
error
successful message
ticket dashboar
valid transaction

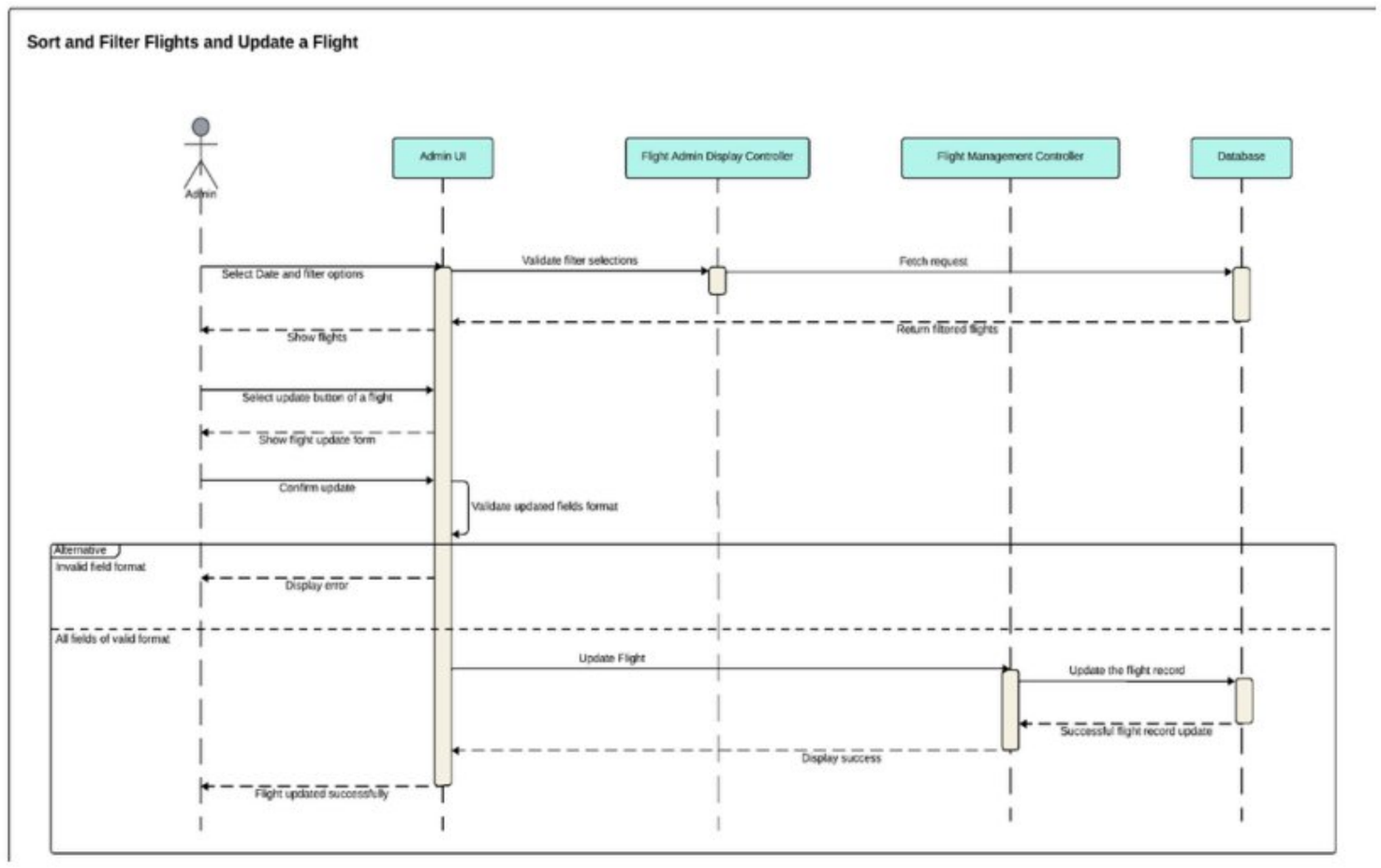
CamScanner

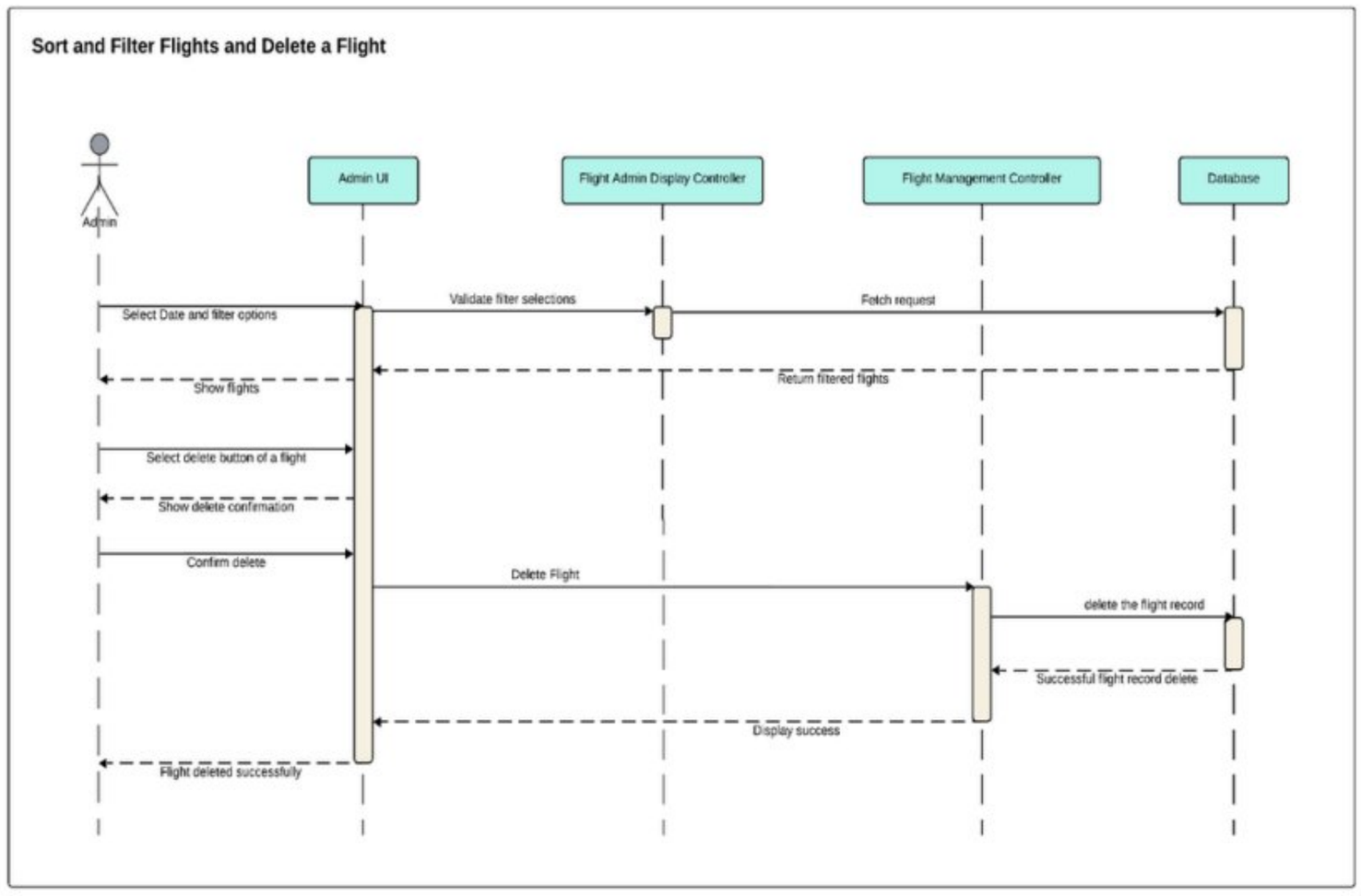


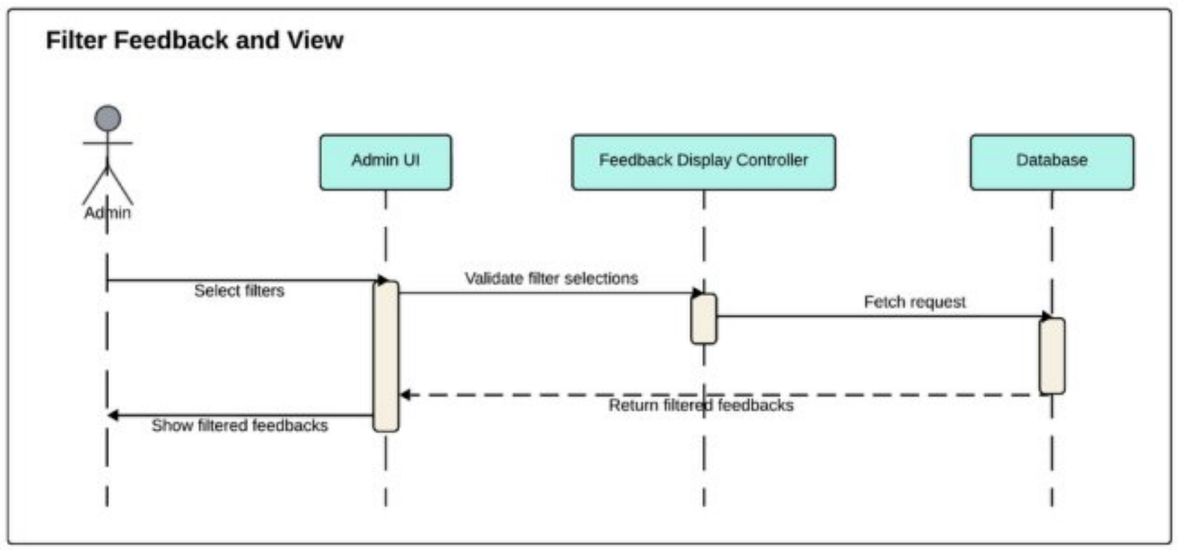
Admin Sequence Diagrams:



CamScanner



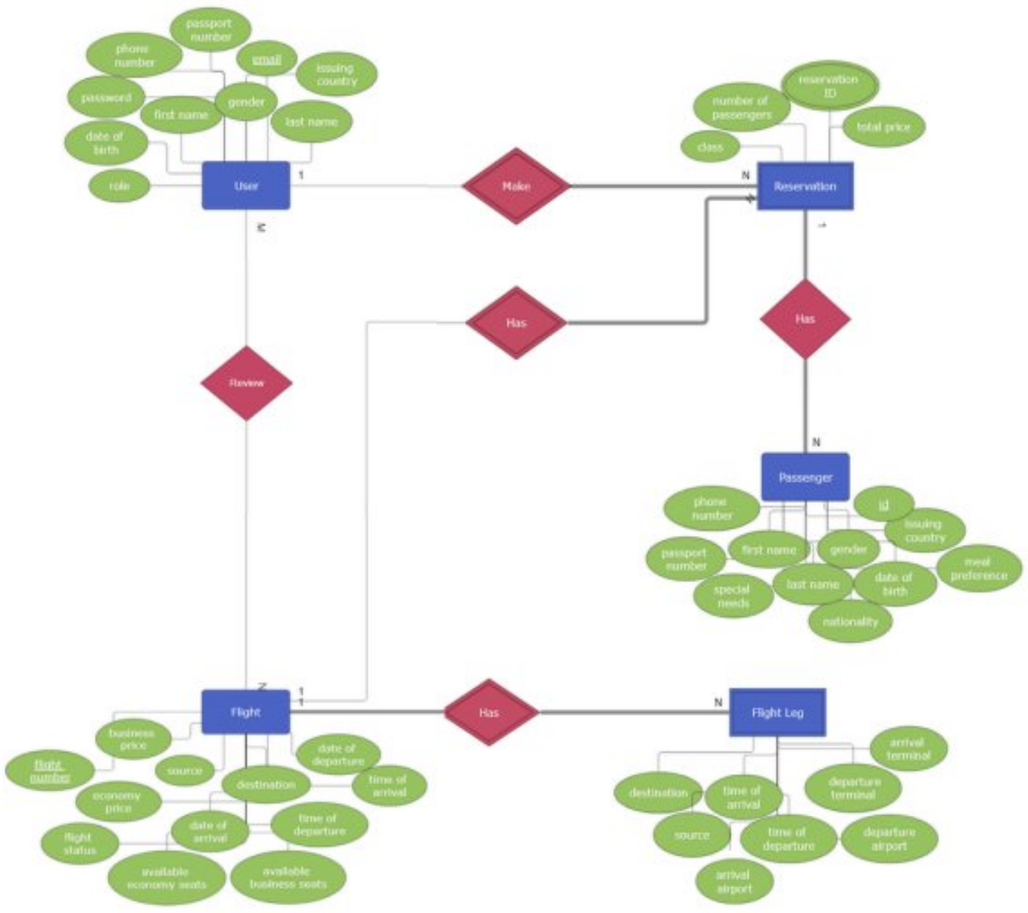




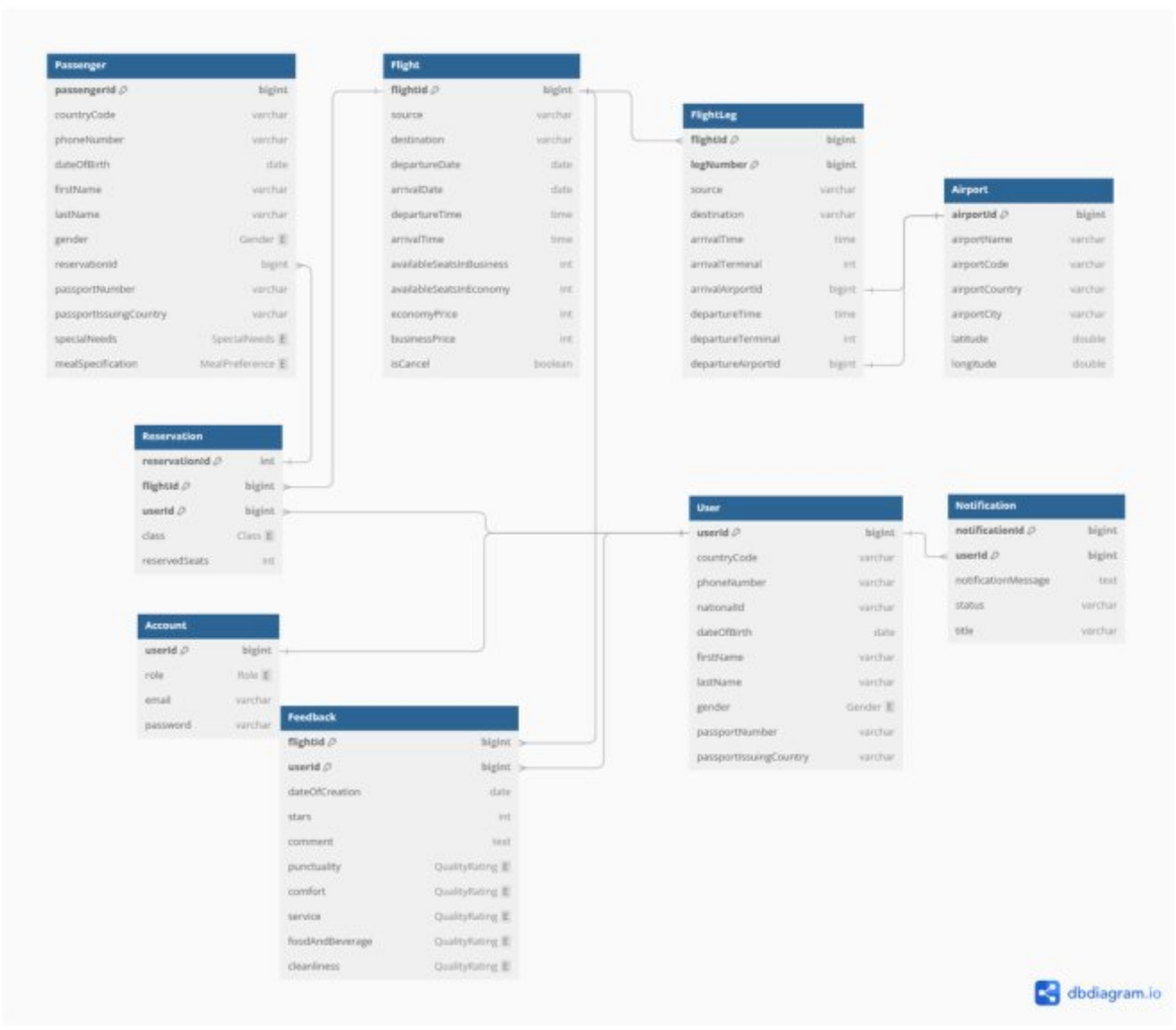
Database Design:

ERD:

CamScanner



Database Schema:



The Features Delivery Milestones:

Milestone1	1. Registration 2. Login 3. Forget Password 4. Change Password 5. All admin listing dashboard
Milestone2	1. User dashboard 2. Booking and searching 3. Add flights
Milestone3	1. Notification 2. Upgrade user 3. Feedback for user

CamScanner

backlog:

Type	Key	Summary	Assignee	Priority	Status	Updated
	SCRUM-23	Flight-Adminstration-System		=	TO DO	Nov 5, 2024, 18:42
	SCRUM-22	Develop Password Recovery System		=	TO DO	Nov 5, 2024, 06:24
	SCRUM-21	Implement Personal Information Update Feature		=	TO DO	Nov 5, 2024, 06:24
	SCRUM-20	Create Admin Account Management System		=	TO DO	Nov 5, 2024, 06:24
	SCRUM-19	Implement Reservation Management for Admins		=	TO DO	Nov 6, 2024, 05:35
	SCRUM-18	Develop Admin Flight Management System		=	TO DO	Nov 6, 2024, 05:33
	SCRUM-17	Develop Email Confirmation System		=	TO DO	Nov 5, 2024, 06:22
	SCRUM-16	Implement Customers Sign-Up Functionality		=	TO DO	Nov 5, 2024, 06:22
	SCRUM-15	Client-Account		=	TO DO	Nov 5, 2024, 18:42
	SCRUM-14	Flight Selection		=	TO DO	Nov 5, 2024, 06:01
	SCRUM-13	Flight Info		=	TO DO	Nov 5, 2024, 06:01
	SCRUM-12	Ticket Cancellation and Refund Process		=	TO DO	Nov 5, 2024, 05:54
	SCRUM-11	Payment Processing Integration		=	TO DO	Nov 5, 2024, 05:54

18 items

Synced just now

CamScanner